Lloyds Banking Group – Task 2 Customer Churn Prediction Model

1. Import Libraries

In [1]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import cross_val_score

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
    roc_curve
)

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

pd.set_option('display.max_columns', None)
```

2. Load Dataset

In [3]:
```
file_path ="../Loyds/smart_bank_churn_project/data/Customer_Churn_Data_Large (1).xl

customer_demo = pd.read_excel(file_path, sheet_name='Customer_Demographics')
transaction_history = pd.read_excel(file_path, sheet_name='Transaction_History')
customer_service = pd.read_excel(file_path, sheet_name='Customer_Service')
online_activity = pd.read_excel(file_path, sheet_name='Online_Activity')
churn_status = pd.read_excel(file_path, sheet_name='Churn_Status')
```

3. Aggregate Transaction Data

In [4]:
```
transaction_agg = transaction_history.groupby('CustomerID').agg({
    'AmountSpent': ['sum', 'mean', 'count']
}).reset_index()

transaction_agg.columns = [
    'CustomerID',
    'TotalSpent',
```

```
    'AverageSpent',
    'TransactionCount'
]

transaction_agg.head()
```

Out[4]:

	CustomerID	TotalSpent	AverageSpent	TransactionCount
0	1	416.50	416.50000	1
1	2	1547.42	221.06000	7
2	3	1702.98	283.83000	6
3	4	917.29	183.45800	5
4	5	2001.49	250.18625	8

4. Aggregate Customer Service Data

In [5]:

```
service_agg = customer_service.groupby('CustomerID').agg({
    'InteractionID': 'count'
}).reset_index()

service_agg.columns = [
    'CustomerID',
    'ServiceInteractionCount'
]

service_agg.head()
```

Out[5]:

	CustomerID	ServiceInteractionCount
0	1	1
1	2	1
2	3	1
3	4	2
4	6	1

5. Merge All Datasets

In [6]:

```
merged_data = customer_demo.merge(
    transaction_agg,
    on='CustomerID',
    how='left'
)

merged_data = merged_data.merge(
    service_agg,
    on='CustomerID',
```

```
    how='left'
)

merged_data = merged_data.merge(
    online_activity,
    on='CustomerID',
    how='left'
)

merged_data = merged_data.merge(
    churn_status,
    on='CustomerID',
    how='left'
)

merged_data.head()
```

Out[6]:

	CustomerID	Age	Gender	MaritalStatus	IncomeLevel	TotalSpent	AverageSpent	Trans
0	1	62	M	Single	Low	416.50	416.50000	
1	2	65	M	Married	Low	1547.42	221.06000	
2	3	18	M	Single	Low	1702.98	283.83000	
3	4	21	M	Widowed	Low	917.29	183.45800	
4	5	21	M	Divorced	Medium	2001.49	250.18625	

In [ ]:
```
6. Handle Missing Values
```

In [7]:
```
merged_data['ServiceInteractionCount'] = (
    merged_data['ServiceInteractionCount'].fillna(0)
)
```

7. Encode Categorical Variables

In [8]:
```
categorical_columns = [
    'Gender',
    'MaritalStatus',
    'IncomeLevel',
    'ServiceUsage'
]

merged_data_encoded = pd.get_dummies(
    merged_data,
    columns=categorical_columns,
    drop_first=True
)

merged_data_encoded.head()
```

Out[8]:

	CustomerID	Age	TotalSpent	AverageSpent	TransactionCount	ServiceInteractionCount
0	1	62	416.50	416.50000	1	1.0
1	2	65	1547.42	221.06000	7	1.0
2	3	18	1702.98	283.83000	6	1.0
3	4	21	917.29	183.45800	5	2.0
4	5	21	2001.49	250.18625	8	0.0

8. Drop Unnecessary Columns

In [9]:
```
merged_data_encoded = merged_data_encoded.drop(
    columns=['CustomerID', 'LastLoginDate']
)

merged_data_encoded.head()
```

Out[9]:

	Age	TotalSpent	AverageSpent	TransactionCount	ServiceInteractionCount	LoginFreque
0	62	416.50	416.50000	1	1.0	
1	65	1547.42	221.06000	7	1.0	
2	18	1702.98	283.83000	6	1.0	
3	21	917.29	183.45800	5	2.0	
4	21	2001.49	250.18625	8	0.0	

9. Define Features and Target

In [10]:
```
X = merged_data_encoded.drop('ChurnStatus', axis=1)

y = merged_data_encoded['ChurnStatus']
```

10. Train Test Split

In [11]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
```

```
    stratify=y
)
```

11. Feature Scaling

In [12]:
```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

12. Logistic Regression Model

In [13]:
```
log_model = LogisticRegression()

log_model.fit(X_train, y_train)
```

Out[13]:
```
▾ LogisticRegression
LogisticRegression()
```

13. Predictions

In [14]:
```
y_pred = log_model.predict(X_test)

y_prob = log_model.predict_proba(X_test)[:,1]
```

14. Model Evaluation

In [15]:
```
accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(y_test, y_pred)

recall = recall_score(y_test, y_pred)

f1 = f1_score(y_test, y_pred)

roc_auc = roc_auc_score(y_test, y_prob)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("ROC AUC Score:", roc_auc)
```

```
Accuracy: 0.795
Precision: 0.0
Recall: 0.0
F1 Score: 0.0
ROC AUC Score: 0.4844301273201411
```

```
c:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

15. Classification Report

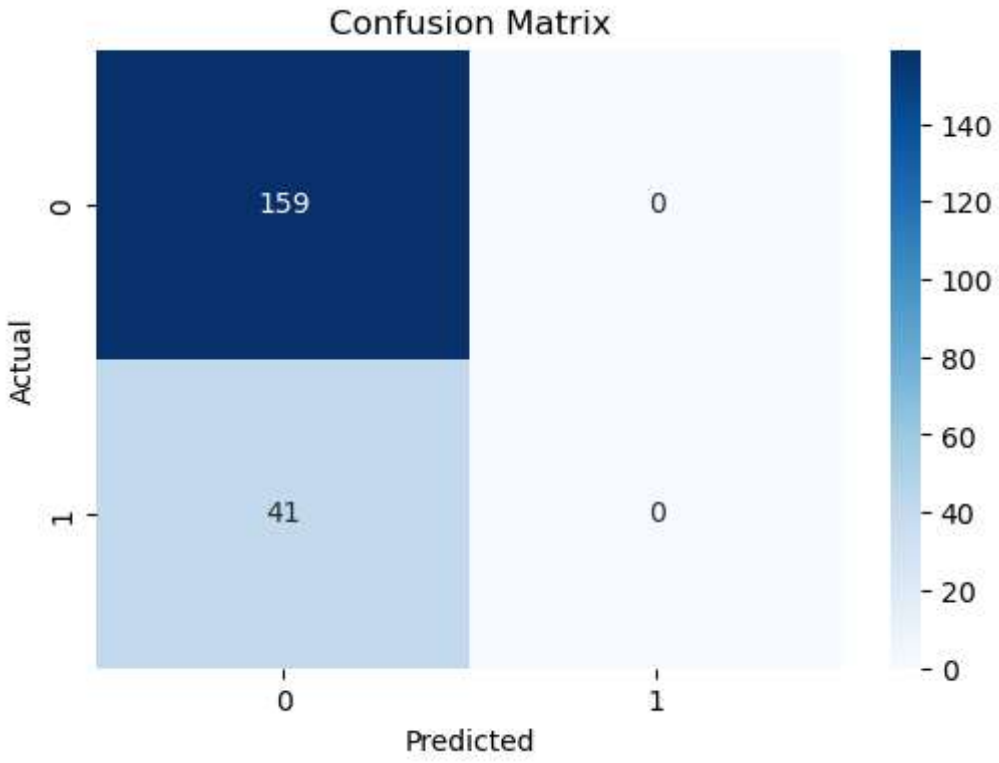
In [16]:
```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.80      1.00      0.89       159
           1       0.00      0.00      0.00        41

    accuracy                           0.80       200
   macro avg       0.40      0.50      0.44       200
weighted avg       0.63      0.80      0.70       200
```

```
c:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
c:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
c:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

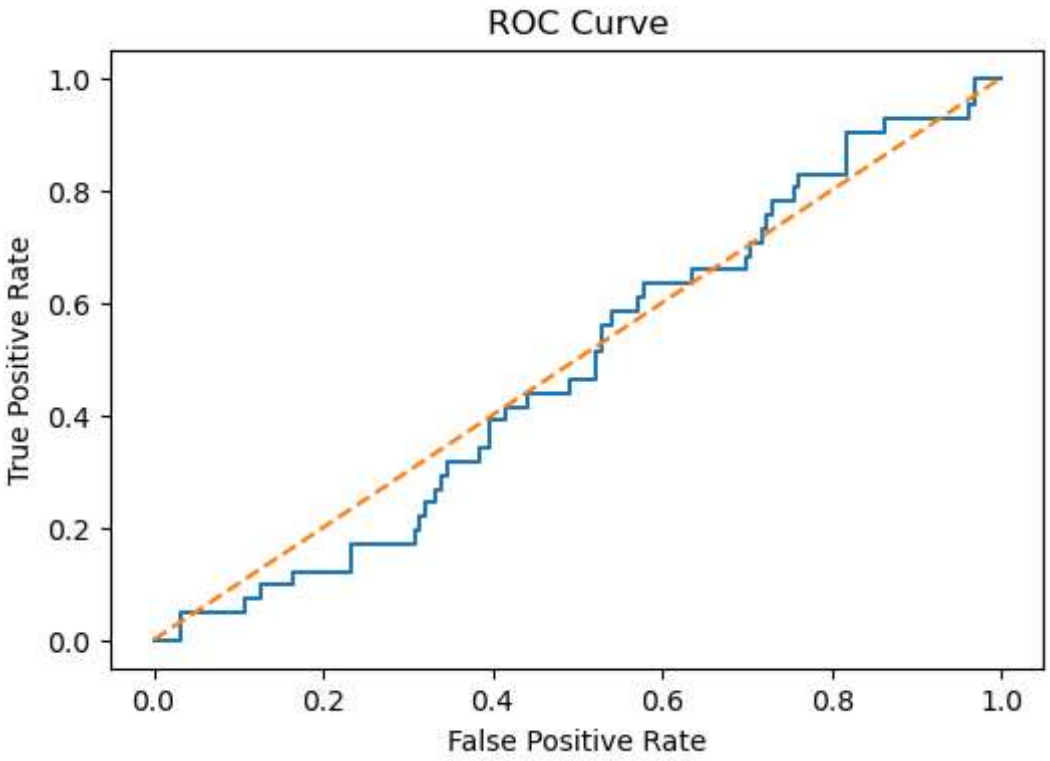
16. Confusion Matrix

In [17]:
```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,4))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.title("Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```



17. ROC Curve

In [18]:
```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)

plt.figure(figsize=(6,4))

plt.plot(fpr, tpr)

plt.plot([0,1], [0,1], linestyle='--')

plt.title("ROC Curve")

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.show()
```



18. Cross Validation

In [19]:
```
cv_scores = cross_val_score(
    log_model,
    X_train,
    y_train,
    cv=5,
    scoring='f1'
)

print("Cross Validation F1 Scores:", cv_scores)

print("Average F1 Score:", cv_scores.mean())
```

```
Cross Validation F1 Scores: [0. 0. 0. 0. 0.]
Average F1 Score: 0.0
```

19. Hyperparameter Tuning

In [20]:
```
param_grid = {
    'C': [0.01, 0.1, 1, 10],
    'solver': ['liblinear']
}

grid_search = GridSearchCV(
    LogisticRegression(),
    param_grid,
    cv=5,
    scoring='f1'
)
```

```
grid_search.fit(X_train, y_train)

print("Best Parameters:", grid_search.best_params_)
```

```
Best Parameters: {'C': 0.01, 'solver': 'liblinear'}
```

20. Feature Importance

In [21]:
```
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': log_model.coef_[0]
})

feature_importance = feature_importance.sort_values(
    by='Coefficient',
    ascending=False
)

feature_importance.head(10)
```

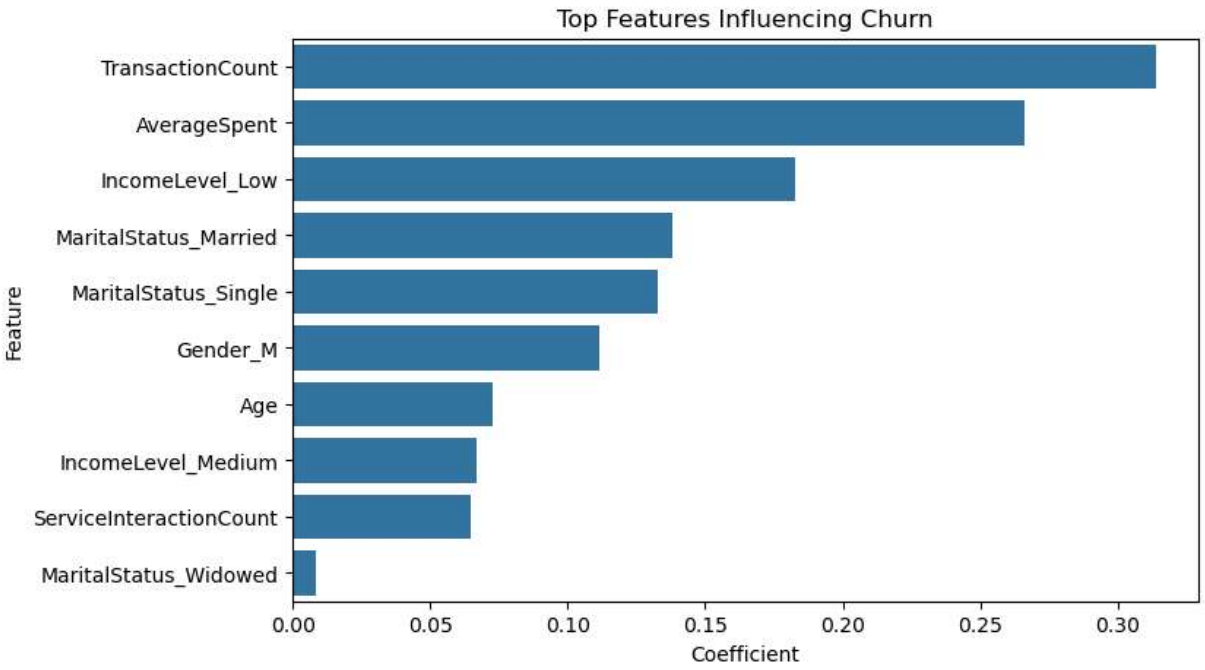
Out[21]:

	Feature	Coefficient
3	TransactionCount	0.313715
2	AverageSpent	0.266173
10	IncomeLevel_Low	0.182562
7	MaritalStatus_Married	0.138163
8	MaritalStatus_Single	0.132993
6	Gender_M	0.111281
0	Age	0.072956
11	IncomeLevel_Medium	0.066643
4	ServiceInteractionCount	0.064884
9	MaritalStatus_Widowed	0.008182

21. Feature Importance Graph

In [22]:
```
top_features = feature_importance.head(10)

plt.figure(figsize=(8,5))

sns.barplot(
    x='Coefficient',
    y='Feature',
    data=top_features
)
```

```
plt.title("Top Features Influencing Churn")

plt.show()
```



22. Business Recommendations

In [23]:
```
print("""
Business Recommendations:

1. Identify customers with low login frequency.
2. Target high-risk customers with personalised offers.
3. Improve customer service response quality.
4. Monitor spending behaviour and engagement levels.
5. Use predictive insights for retention campaigns.
""")
```

```
Business Recommendations:

1. Identify customers with low login frequency.
2. Target high-risk customers with personalised offers.
3. Improve customer service response quality.
4. Monitor spending behaviour and engagement levels.
5. Use predictive insights for retention campaigns.
```

23. Conclusion

In [24]:
```
print("""
Conclusion:

A Logistic Regression model was built to predict customer churn.
The model was evaluated using Accuracy, Precision, Recall,
F1 Score, ROC-AUC, and Confusion Matrix analysis.
```

```
The model provides interpretable insights into customer churn
behaviour and can help Lloyds Banking Group improve customer
retention strategies.
""")
```

```
Conclusion:

A Logistic Regression model was built to predict customer churn.
The model was evaluated using Accuracy, Precision, Recall,
F1 Score, ROC-AUC, and Confusion Matrix analysis.

The model provides interpretable insights into customer churn
behaviour and can help Lloyds Banking Group improve customer
retention strategies.
```